

CSE535 Project Report: Hand Gesture Recognition

A. Approach to the Problem

The task is to develop a gesture recognition application that can accurately classify hand gestures captured in videos into one of 17 predefined classes. These gestures include digits (0–9) and home automation actions like LightOn, FanOff, SetThermo, etc. The goal is to train a model using provided videos and then predict the gesture class for unseen test videos. The evaluation is based on both correct implementation (compilation) and prediction accuracy.

Main Challenges

- Low accuracy when training on self-recorded videos, necessitating a switch to expert-provided videos.
- Issues dealing with mislabeled or corrupted video data.
- Formatting the Results.csv file correctly for the autograder to process.
- Matching test video names to the correct gesture labels.
- Inconsistent video naming required hardcoding of label mappings.
- General challenges related to labeling and directory structure.

Project Steps

Frame Extraction

One representative frame was extracted from each training and test video using a custom `frameExtractor()` function.

Feature Extraction

A pre-trained Convolutional Neural Network (CNN), provided via the `HandShapeFeatureExtractor` class, was used to extract a fixed-length feature vector from each frame.

Label Mapping

Each training video filename was parsed to determine its label. A `label_map` dictionary was manually created to map these label strings to integer indices due to inconsistencies in file naming.

Prediction via Cosine Similarity

Instead of a traditional classifier, cosine similarity was computed between each test feature and the full training set. The closest match determined the predicted label.

Results File Generation

The predictions were written to a Results.csv file in the format expected by the autograder.

B. Solution for the Problem

All logic was implemented in the main.py file. The script performs training, prediction, and results generation.

Switching to Expert Videos

Initially, I used my own gesture recordings for training. However, upon manually inspecting and comparing the extracted features from training and test data, I noticed that the matching scores were very low, indicating poor similarity. This was likely due to factors such as:

- Poor lighting conditions
- Varying backgrounds
- Inconsistent or inaccurate hand gestures

As a result, I decided to replace the training set with expert-provided videos, which were cleaner, more consistent, and better aligned with the test data. This significantly improved the feature matching and overall prediction accuracy.

Labeling and Hardcoding

Due to the inconsistent and mixed naming of gesture files (e.g., Num7, FanDown, 0, IncreaseFanSpeed), I created a hardcoded label_map dictionary. This ensured consistent labeling of gestures across training and testing, which was crucial for model evaluation.

Prediction Using Cosine Similarity

Although I initially attempted classification using an SVM model, it performed poorly. I then adopted a cosine similarity approach:

```
similarities = cosine_similarity(test_feature, train_features)[0]  
predicted_label = train_labels[np.argmax(similarities)]
```

This method provided a more robust prediction by comparing feature vectors directly rather than relying on a trained model.

Result Formatting for Autograder

One major challenge was ensuring the result format matched autograder requirements. Initially, I generated a 51×2 CSV with both video name and label, which the autograder could not process. Eventually, I resolved the issue by:

- Outputting only the predicted labels (1 column × 51 rows)
- Ensuring proper order and line count
- Removing unnecessary file extensions or video names from predictions

After this correction, the autograder was able to process the output correctly.

Conclusion

This project provided hands-on experience with feature extraction, model training, and result evaluation in gesture recognition. Though I encountered several challenges—especially with training data quality, filename mismatches, and result formatting—careful debugging and switching to expert data led to a working, gradable solution.